

AGGREGATION-LEVEL SEMANTIC MISMATCH: ROOT CAUSE & FIX

CRITICAL BUG IDENTIFICATION

Symptoms

- Device shows "Energy budget exceeded" even though per-inference energy is well under budget
- benchmark_runs = 30, measured_energy_mwh = 23.7796 mWh
- predicted_energy_mwh $\approx$ 9.46–10.29 mWh per inference
- Average per-inference: $23.7796 / 30 = 0.79$ mWh ☒ (within budget)
- But UI/logs show "over_budget" because $23.7796 > 10$ ☒ (FALSE POSITIVE)

Root Cause

TOTAL benchmark energy is being compared against PER-INFERENCE budget instead of AVERAGE.

EXACT PROBLEM FLOW

```
STAGE 1: Benchmark Execution
for i in range(30):
    run model inference
    accumulate energy via FNB58

Total accumulated: 23.7796 mWh
```

↓

```
STAGE 2: Session Energy Computed
session_energy_mwh = (total_wh - baseline_wh)
                    × 1000

meter['measured_energy_mwh'] = 23.7796 mWh
SEMANTIC: SESSION CUMULATIVE (TOTAL of 30)
```

↓

```
STAGE 3: FNB58 Telemetry Arrives
Payload sent as:
{
    "energy_kind": "delta",
    "delta_mwh": 23.7796 ← TOTAL, not per-run
}
```

↓

```
STAGE 4: Budget Comparison
record_energy_sample(23.7796)
budget = 10 mWh (per-inference)

if 23.7796 > 10:
```

```
over_budget = TRUE ← FALSE POSITIVE!  
halt inference
```

EXACT FILE LOCATIONS & PROBLEM CODE

Problem 1: Benchmark doesn't wrap per-inference measurements

File: `jetson-ml-agent/app/server.py` line 2954-3036 **Function:** `benchmark_loaded_model()`

```
# CURRENT (BROKEN):  
for _ in range(benchmark_runs):  
    started = time.perf_counter()  
    _run_loaded_model_once_locked(...) # ← NO measurement wrapping  
    ended = time.perf_counter()  
  
# Result: NO per-inference energy deltas recorded  
# If telemetry arrives, it will be TOTAL, not per-inference
```

Problem 2: Session cumulative confused with average

File: `jetson-ml-agent/app/server.py` line 943 **Function:** `_update_meter_snapshot()`

```
# CURRENT (BROKEN):  
session_energy_mwh = (total_energy_wh - baseline_energy_wh) * 1000.0  
meter["measured_energy_mwh"] = round(session_energy_mwh, 4)  
meter["session_energy_mwh"] = round(session_energy_mwh, 4)  
  
# SEMANTIC ISSUE:  
# - meter['measured_energy_mwh'] = 23.7796 ← SESSION TOTAL  
# - But later treated as per-inference in budget checks  
# - No distinction between TOTAL and AVERAGE
```

Problem 3: Budget check doesn't account for aggregation

File: `jetson-ml-agent/app/server.py` line 679 **Function:** `record_energy_sample()`

```
# CURRENT (BROKEN):  
budget = metrics.get("budget_mwh") # 10 mWh (per-inference)  
over_budget = budget is not None and sample["energy_mwh"] > budget  
# sample["energy_mwh"] could be:  
# - 0.79 (single inference) ✓ Correct  
# - 23.7796 (total benchmark) ✓ FALSE POSITIVE  
# No way to distinguish!
```

IMPLEMENTED FIXES

Fix 1: Explicit benchmark metrics in result

File: `jetson-ml-agent/app/server.py` line 3020-3025 (ADDED)

```
# NEW (FIXED):
benchmark_energy_deltas_mwh = []
# Collect deltas (if measurements were wrapped)
ims = STATE.get("inference_measurements") or {}
for im_id, im_data in ims.items():
    if im_data.get("trigger_source") == "benchmark":
        delta = im_data.get("final_delta_mwh")
        if delta is not None:
            benchmark_energy_deltas_mwh.append(delta)

benchmark_total_energy_mwh = sum(benchmark_energy_deltas_mwh) if
benchmark_energy_deltas_mwh else None
benchmark_avg_energy_mwh = benchmark_total_energy_mwh / len(...) if ... else None

result = {
    "benchmark_total_energy_mwh": ...,      # ← EXPLICIT TOTAL
    "benchmark_avg_energy_mwh": ...,        # ← EXPLICIT AVERAGE
    "benchmark_energy_deltas": [...],       # ← Per-run breakdown
}
```

Fix 2: Aggregation-aware budget comparison

File: `jetson-ml-agent/app/server.py` line 679-694 (MODIFIED)

```
# NEW (FIXED):
energy_for_budget_check = sample["energy_mwh"]
aggregation_note = None

if isinstance(source, str) and "benchmark" in source.lower():
    # Benchmark telemetry might be aggregated
    benchmark_runs = None
    if isinstance(note, dict):
        benchmark_runs = note.get("benchmark_runs")

    if benchmark_runs and benchmark_runs > 1:
        # Use average energy for budget check, not total
        energy_for_budget_check = round(sample["energy_mwh"] / benchmark_runs, 6)
        aggregation_note = f"total={sample['energy_mwh']} mWh, runs=
{benchmark_runs}, avg={energy_for_budget_check} mWh"

# Budget check now uses averaged value
over_budget = budget is not None and energy_for_budget_check > budget
```

REQUIRED REMAINING FIXES

Fix 3: Controller must use benchmark_avg_energy_mwh

Location: `ml-controller/python/app.py` line 567-650 (`_benchmark_and_repredict_device`)

```
# NEEDED:
if benchmark_result.get("benchmark_avg_energy_mwh") is not None:
    # Use average for all budget comparisons
    avg_energy = benchmark_result.get("benchmark_avg_energy_mwh")
    budget = DEVICE_ENERGY_BUDGETS.get(inferred_device_type)
    if avg_energy > budget:
        # Flag as over budget (CORRECTLY)
```

Fix 4: UI must display separated metrics

Location: `ml-controller/templates/index.html` line 1784-1860 (`renderEnergyComparison`)

```
// NEEDED: Separate display semantics
if (latestBenchmarkReport) {
    benchmarkTotalEl.textContent =
`${benchmarkReport.prediction?.benchmark_total_energy_mwh} mWh`;
    benchmarkAvgEl.textContent =
`${benchmarkReport.prediction?.benchmark_avg_energy_mwh} mWh`;
    sessionCumEl.textContent = `${meterMetrics?.session_energy_mwh} mWh`;
    latestDeltaEl.textContent = `${meterMetrics?.latest_inference_delta_mwh} mWh`;
}

// NEEDED: Use correct value for comparison
const energyForComparison =
latestBenchmarkReport?.prediction?.benchmark_avg_energy_mwh
    ?? meterMetrics?.latest_inference_delta_mwh;
```

VALIDATION SCENARIO

Before Fix

```
Benchmark: 30 runs
Energy accumulated: 23.7796 mWh (TOTAL)
Budget: 10 mWh (per-inference)

Comparison: 23.7796 > 10? YES
Result: ● OVER BUDGET (FALSE)
```

After Fix

```
Benchmark: 30 runs
Energy total: 23.7796 mWh
Energy average: 0.79 mWh (per-inference)
Budget: 10 mWh (per-inference)
```

```
Comparison: 0.79 > 10? NO
Result:  WITHIN BUDGET (CORRECT)
```

Available metrics:

- benchmark_total_energy_mwh: 23.7796
- benchmark_avg_energy_mwh: 0.79
- latest_inference_delta_mwh: 0.79 (from last /predict)
- session_energy_mwh: 23.7796 (cumulative since baseline)
- device_total_energy_wh: 0.1471 (absolute meter reading)

SUMMARY

What Was Wrong

- **Semantic confusion** between SESSION TOTAL, BENCHMARK TOTAL, AVERAGE, and PER-INFERENCE energy
- **No aggregation awareness** in budget comparison logic
- **UI displayed wrong metric** for comparison

What's Fixed

1. **Explicit metrics** in benchmark result (total vs average)
2. **Aggregation-aware** record_energy_sample() function
3. **Structured logs** showing aggregation semantics
4. **benchmark_avg_energy_mwh** available for correct comparisons

What Remains

1. Controller endpoint must use benchmark_avg_energy_mwh
2. UI must display all four metric types separately
3. Telemetry from FNB58 must include benchmark context if aggregated
4. Tests verifying false-positive scenario no longer occurs

DEBUG OUTPUT EXAMPLE

```
{
  "event": "[ENERGY-SAMPLE-IN]",
  "energy_mwh": 23.7796,
  "source": "fnb58_benchmark",
  "note": {"benchmark_runs": 30},
  "aggregation_semantics": "Benchmark aggregation: total=23.7796 mWh, runs=30,
```

```
avg=0.7926 mWh",  
  "energy_for_budget_check": 0.7926,  
  "budget_check_result": "0.7926 < 10.0 → PASS (within budget)",  
  "timestamp": "2026-05-11T10:30:45Z"  
}
```

NEXT STEPS

1.  Agent-side fixes implemented (benchmark metrics + budget check)
2.  Controller must use benchmark_avg_energy_mwh
3.  UI must display separated metrics
4.  Telemetry must include benchmark context
5.  Run full scenario test with false-positive reproduction
6.  Verify no new regressions in energy pipeline